

Binary



Contents

1 영상의 이진화

1.1 영상의 이진화

1.2 그레이 이미지의 이진화

1.3 threshold 함수

1.4 threshold의 type

1.5 이진화 동작

1.6 코드

2 Otsu

2.1 자동으로 threshold를 잡고 싶다면

2.2 Otsu 이진화

2.3 Otsu 이진화 결과

2.4 코드

3 지역 이진화

3.1 균일하지 않은 조명에서 일괄적인 threshold 적용은 문제가 된다

3.2 많이 사용하는 방법은 분할시켜서 각각 이진화

3.3 적응형 이진화 함수

3.4 코드

4 픽셀 라벨링

4.1 라벨링

4.2 예제

4.3 라벨링 함수

4.4 정보를 보다 많이 반환하는 함수

4.5 이런 결과가 나타남

4.6 코드

4.7 이번에는~

4.8 코드

5 외곽선 검출

5.1 외곽선 검출 함수

5.2 외곽선 그리는 함수

5.3 외곽선 검출 예제

5.4 결과

5.5 계층 정보 없는 외곽선

5.6 이렇게

5.7 외곽선 관련 함수

5.8 외곽선 길이 구하기

5.9 면적 구하기

5.10 바운딩 박스 구하기

5.11 바운딩 서클 구하기

5.12 외곽선 근사화

5.13 컨벡스 검사

5.14 다각형 검출해보기

5.15 원 판별하기

5.16 코드

5.17 결과

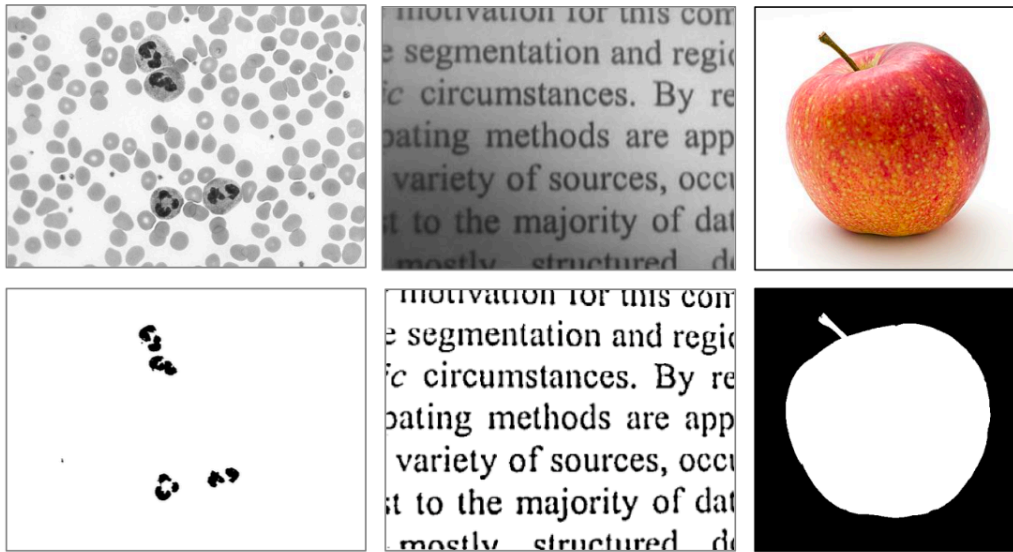
Binary

Exported from Confluence · <https://pinkwink.atlassian.net>

영상의 이진화

영상의 이진화

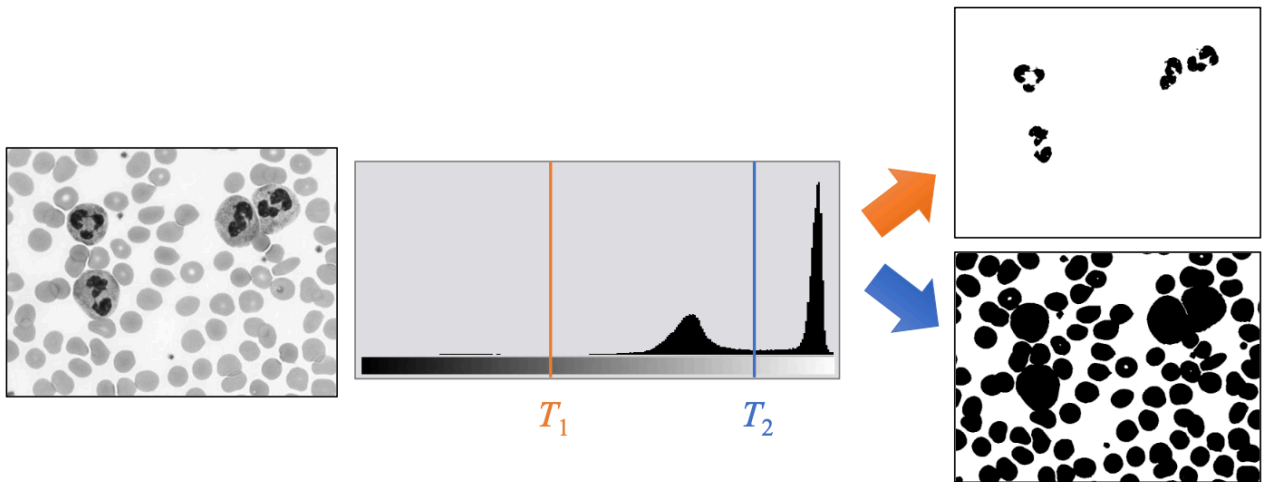
- 영상의 픽셀 값을 0 또는 255(1)로 만드는 연산
 - 배경(background) vs. 객체(object)
 - 관심 영역 vs. 비관심 영역



그레이 이미지의 이진화

$$g(x, y) = \begin{cases} 0 & \text{if } f(x, y) \leq T \\ 255 & \text{if } f(x, y) > T \end{cases}$$

▪ T : 임계값, 문턱치, threshold



threshold 함수

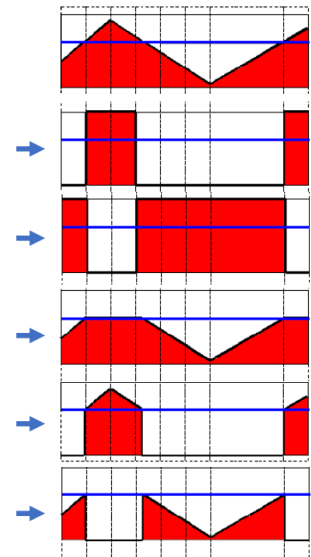
```
cv2.threshold(src, thresh, maxval, type, dst=None) -> retval, dst
```

- src: 입력 영상. 다채널, 8비트 또는 32비트 실수형.
- thresh: 사용자 지정 임계값
- maxval: cv2.THRESH_BINARY 또는 cv2.THRESH_BINARY_INV 방법 사용 시 최댓값. 보통 255로 지정.
- type: cv2.THRESH_로 시작하는 플래그. 임계값 함수 동작 지정 또는 자동 임계값 결정 방법 지정
- retval: 사용된 임계값
- dst: 출력 영상. src와 동일 크기, 동일 타입, 같은 채널 수.

threshold의 type

■ cv2.threshold() 함수 동작 타입

cv2.THRESH_BINARY	$\text{dst}(x, y) = \begin{cases} \text{maxval} & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$
cv2.THRESH_BINARY_INV	$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{maxval} & \text{otherwise} \end{cases}$
cv2.THRESH_TRUNC	$\text{dst}(x, y) = \begin{cases} \text{threshold} & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$
cv2.THRESH_TOZERO	$\text{dst}(x, y) = \begin{cases} \text{src}(x, y) & \text{if } \text{src}(x, y) > \text{thresh} \\ 0 & \text{otherwise} \end{cases}$
cv2.THRESH_TOZERO_INV	$\text{dst}(x, y) = \begin{cases} 0 & \text{if } \text{src}(x, y) > \text{thresh} \\ \text{src}(x, y) & \text{otherwise} \end{cases}$
cv2.THRESH_OTSU	Otsu 알고리즘으로 임계값 자동 결정
cv2.THRESH_TRIANGLE	삼각 알고리즘으로 임계값 자동 결정



이진화 동작

```
src = cv2.imread('cells.png', cv2.IMREAD_GRAYSCALE)

_, dst1 = cv2.threshold(src, 100, 255, cv2.THRESH_BINARY)
_, dst2 = cv2.threshold(src, 210, 255, cv2.THRESH_BINARY)
```



코드

```

import sys
import cv2

src = cv2.imread("./data/cells.jpg", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

_, dst1 = cv2.threshold(src, 100, 255, cv2.THRESH_BINARY)
_, dst2 = cv2.threshold(src, 210, 255, cv2.THRESH_BINARY)

cv2.imshow("src", src)
cv2.imshow("dst1", dst1)
cv2.imshow("dst2", dst2)
cv2.waitKey()
cv2.destroyAllWindows()

```

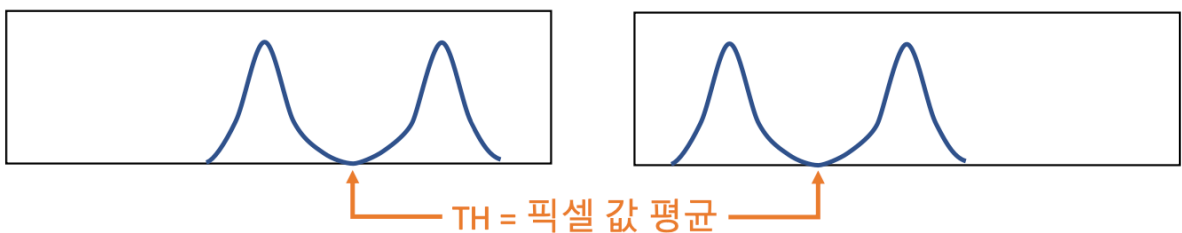
✓ 2.8s

Python

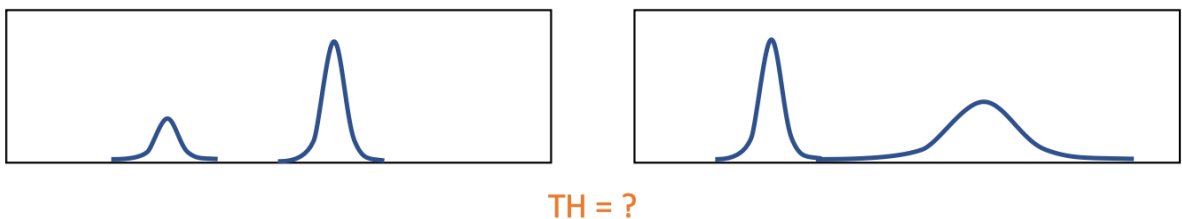
Otsu

자동으로 threshold를 잡고 싶다면

- 영상의 히스토그램이 bimodal이고, 전경&배경 픽셀 분포가 비슷하다면?

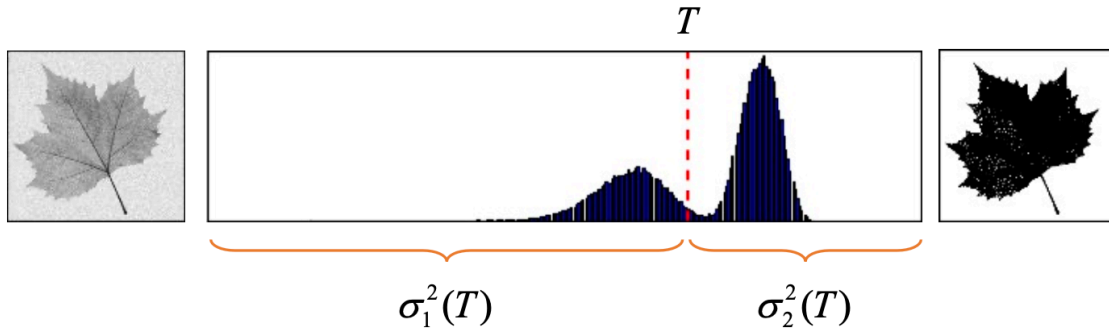


- 히스토그램이 bimodal이지만, 전경&배경 픽셀 분포가 크게 다르다면?



Otsu 이진화

- 입력 영상이 배경(background)과 객체(object) 두 개로 구성되어 있다고 가정
→ Bimodal histogram
- 임의의 임계값 T 에 의해 나뉘는 두 픽셀 분포 그룹의 분산이 최소가 되는 T 를 선택
- 일종의 최적화 알고리즘(optimization algorithm)



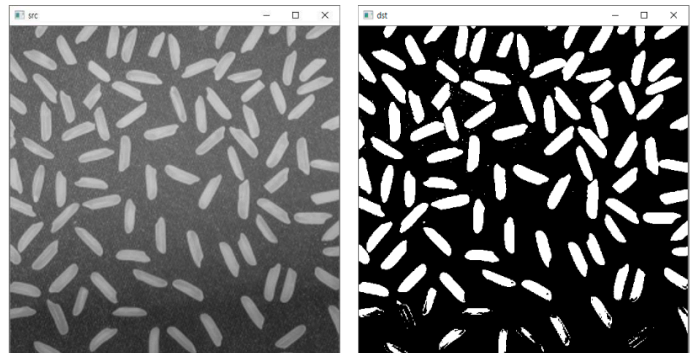
Otsu 이진화 결과

```
src = cv2.imread('rice.png', cv2.IMREAD_GRAYSCALE)

th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)
print("otsu's threshold:", th) # 131
```

```
관리자: C:\Windows\System32\cmd.exe - python otsu.py

c:\coding\python\opencv\week4>python otsu.py
otsu's threshold: 131.0
```



코드


```
import sys
import cv2

src = cv2.imread("./data/rice.jpg", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

th, dst = cv2.threshold(src, 0, 255, cv2.THRESH_BINARY | cv2.THRESH_OTSU)
print("otsu's threshold:", th)

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

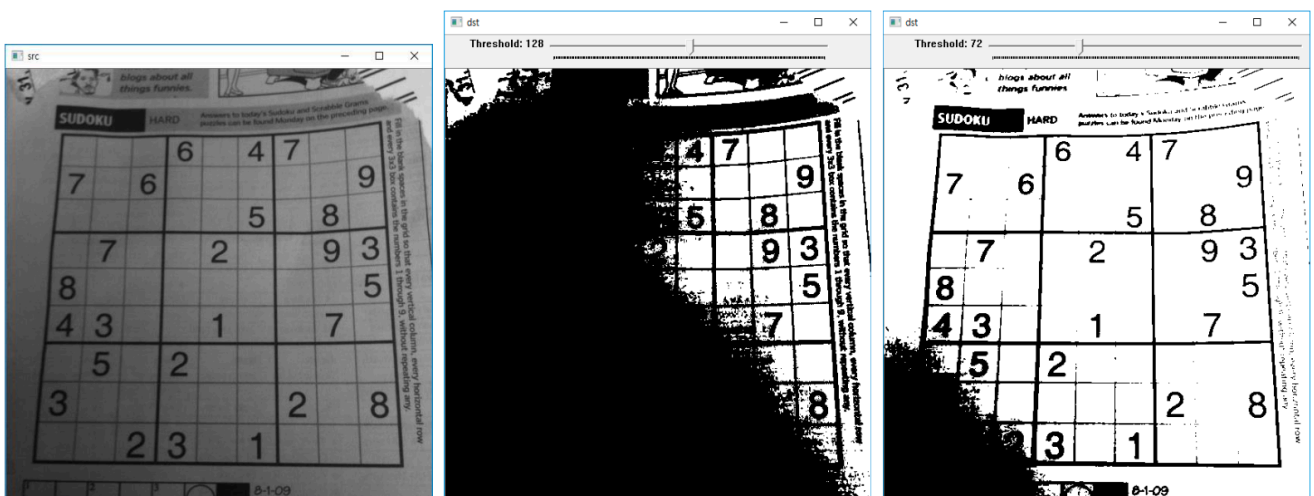
✓ 1.6s

Python

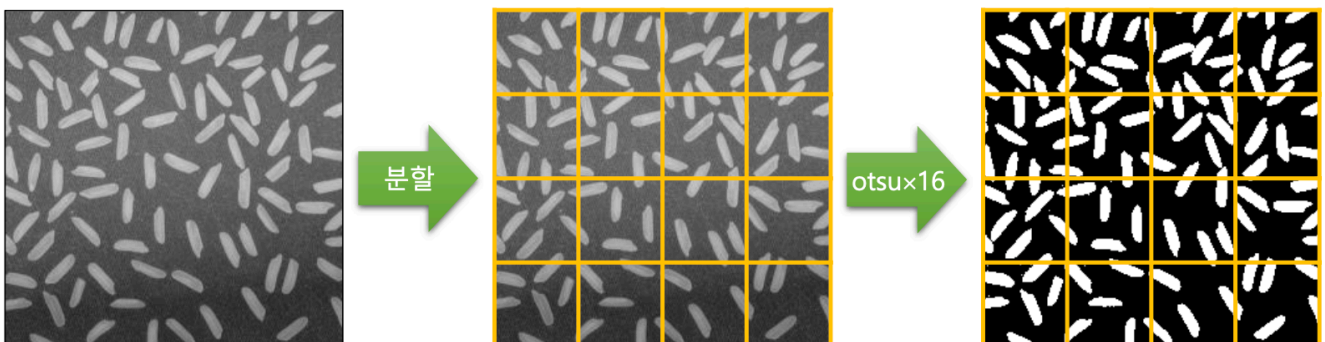
otsu's threshold: 132.0

지역 이진화

균일하지 않은 조명에서 일괄적인 threshold 적용은 문제가 된다



많이 사용하는 방법은 분할시켜서 각각 이진화



적응형 이진화 함수

```
cv2.adaptiveThreshold(src, maxValue, adaptiveMethod,
                     thresholdType, blockSize, C, dst=None) -> dst
```

- src: 입력 영상. 그레이스케일 영상.
- maxValue: 임계값 함수 최대값. 보통 255.
- adaptiveMethod: 블록 평균 계산 방법 지정. cv2.ADAPTIVE_THRESH_MEAN_C는 산술 평균, cv2.ADAPTIVE_THRESH_GAUSSIAN_C는 가우시안 가중치 평균
- thresholdType: cv2.THRESH_BINARY 또는 cv2.THRESH_BINARY_INV 지정
- blockSize: 블록 크기. 3 이상의 홀수.
- C: 블록 내 평균값 또는 블록 내 가중 평균값에서 뺀 값.
(x, y) 픽셀의 임계값으로 $T(x, y) = \mu_B(x, y) - C$ 를 사용

코드

```
import sys
import cv2

src = cv2.imread("./data/sudoku.jpg", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

bsize = 201
dst = cv2.adaptiveThreshold(
    src, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, bsize, 5
)

cv2.imshow("dst", dst)
cv2.imshow("src", src)

cv2.namedWindow("dst")

cv2.waitKey()
cv2.destroyAllWindows()
```

✓ 3.1s

Python

픽셀 라벨링

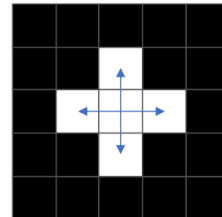
라벨링

■ 레이블링(Labeling)이란?

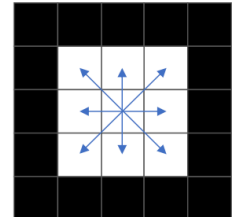
- 동일 객체에 속한 모든 픽셀에 고유한 번호를 매기는 작업
- 일반적으로 이진 영상에서 수행
- OpenCV에는 3.x 버전부터 최신 논문 기반의 레이블링 알고리즘 함수를 제공
- Connected component labeling

■ 픽셀의 연결 관계

- 4-이웃 연결 관계(4-neighbor connectivity)
- 8-이웃 연결 관계(8-neighbor connectivity)

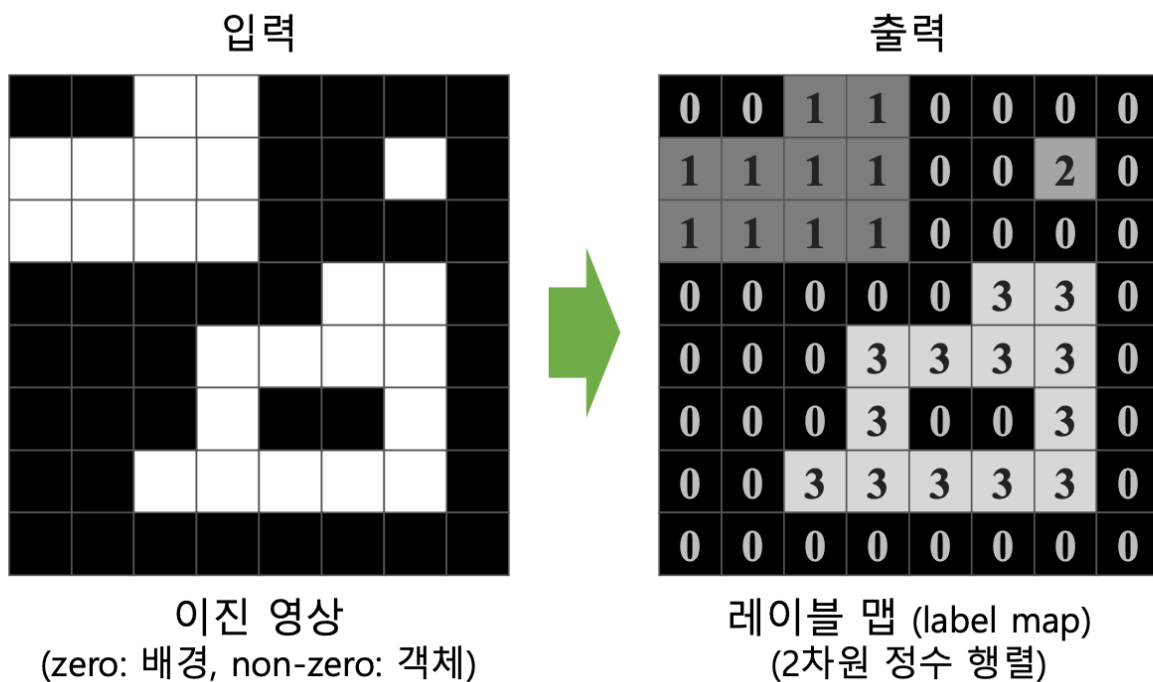


4-이웃 연결 관계



8-이웃 연결 관계

예제



라벨링 함수

```
cv2.connectedComponents(image, labels=None, connectivity=None,
                        ltype=None) -> retval, labels
```

- image: 8비트 1채널 영상
- labels: 레이블 맵 행렬. 입력 영상과 같은 크기. `numpy.ndarray`.
- connectivity: 4 또는 8. 기본값은 8.
- ltype: labels 타입. `cv2.CV_32S` 또는 `cv2.CV_16S`. 기본값은 `cv2.CV_32S`.
- retval: 객체 개수. N을 반환하면 `[0, N-1]`의 레이블이 존재 하며, 0은 배경을 의미. (실제 흰색 객체 개수는 N-1개)

정보를 보다 많이 반환하는 함수

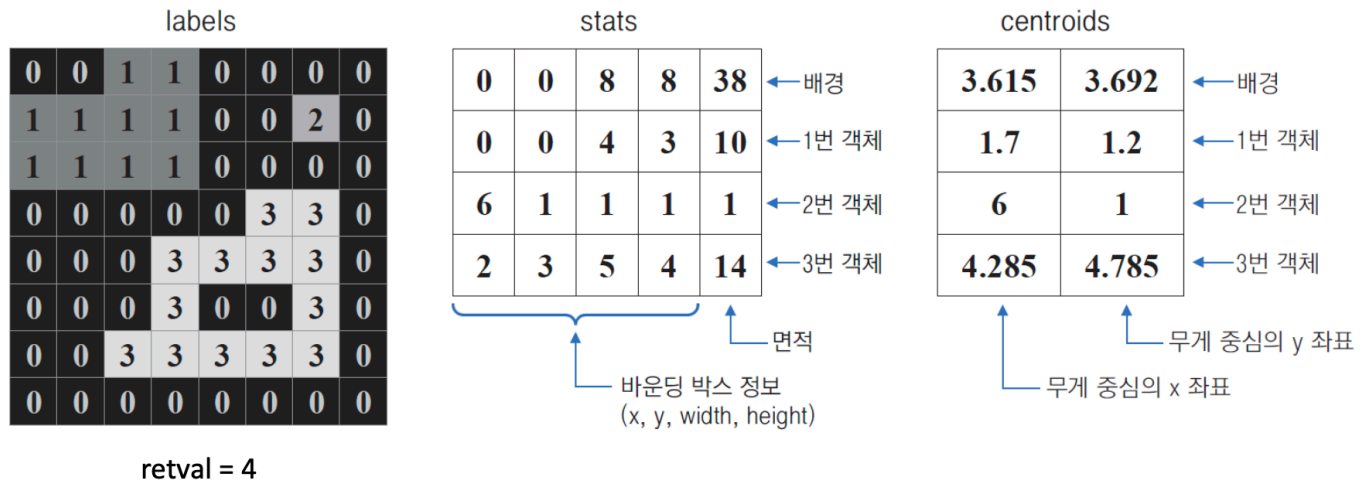
```
cv2.connectedComponentsWithStats(image, labels=None, stats=None,
                                centroids=None, connectivity=None, ltype=None)
-> retval, labels, stats, centroids
```

- image: 8비트 1채널 영상
- labels: 레이블 맵 행렬. 입력 영상과 같은 크기. `numpy.ndarray`.
- stats: 각 객체의 바운딩 박스, 픽셀 개수 정보를 담은 행렬. `numpy.ndarray`. shape=(N, 5), dtype=`numpy.int32`.
- centroids: 각 객체의 무게 중심 위치 정보를 담은 행렬 `numpy.ndarray`. shape=(N, 2), dtype=`numpy.float64`.
- ltype: labels 행렬 타입. `cv2.CV_32S` 또는 `cv2.CV_16S`. 기본값은 `cv2.CV_32S`.

이런 결과가 나타남

■ cv2.connectedComponentsWithStats() 함수 수행 결과의 예

```
retval, labels, stats, centroids = cv2.connectedComponentsWithStats(src)
```



코드

```

import sys
import numpy as np
import cv2

mat = np.array(
    [
        [0, 0, 1, 1, 0, 0, 0, 0],
        [1, 1, 1, 1, 0, 0, 1, 0],
        [1, 1, 1, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 1, 1, 0],
        [0, 0, 0, 1, 1, 1, 1, 0],
        [0, 0, 1, 1, 0, 0, 1, 0],
        [0, 0, 1, 1, 1, 1, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 0],
    ],
    np.uint8,
)

cnt, labels = cv2.connectedComponents(mat)

print("sep:", mat, sep="\n")
print("cnt:", cnt)
print("labels:", labels, sep="\n")

```

✓ 0.0s

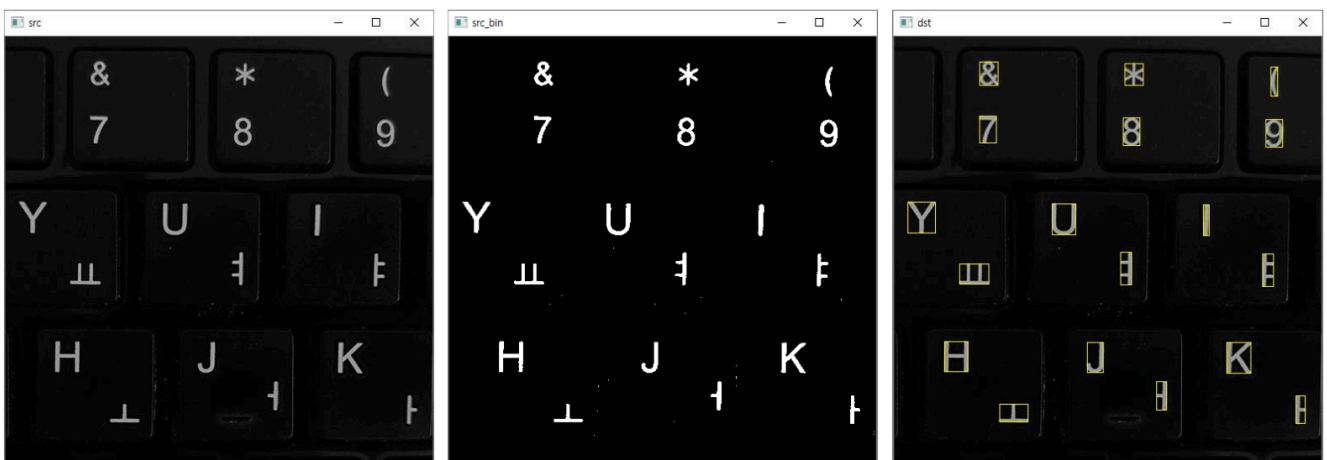
Python

```

sep:
[[0 0 1 1 0 0 0 0]
 [1 1 1 1 0 0 1 0]
 [1 1 1 1 0 0 0 0]
 [0 0 0 0 0 1 1 0]
 [0 0 0 1 1 1 1 0]
 [0 0 1 1 0 0 1 0]
 [0 0 1 1 1 1 1 0]
 [0 0 0 0 0 0 0 0]]
cnt: 4
labels:
[[0 0 1 1 0 0 0 0]
 [1 1 1 1 0 0 2 0]
 [1 1 1 1 0 0 0 0]
 [0 0 0 0 0 3 3 0]
 [0 0 0 3 3 3 3 0]
 [0 0 3 3 0 0 3 0]
 [0 0 3 3 3 3 3 0]
 [0 0 0 0 0 0 0 0]]

```

이번에는~



코드

```
1 import sys
2 import cv2
3
4 src = cv2.imread("./data/keyboard.jpg", cv2.IMREAD_GRAYSCALE)
5
6 if src is None:
7     print("Image load failed!")
8     sys.exit()
9
10 _, src_bin = cv2.threshold(src, 0, 255, cv2.THRESH_OTSU)
11
12 cnt, labels, stats, centroids = cv2.connectedComponentsWithStats(src_bin)
13
14 dst = cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)
15
16 for i in range(1, cnt):
17     (x, y, w, h, area) = stats[i]
18
19     if area < 20:
20         continue
21
22     cv2.rectangle(dst, (x, y, w, h), (0, 255, 255))
23     cv2.putText(
24         dst,
25         str(i),
26         (x, y),
27         cv2.FONT_HERSHEY_SIMPLEX,
28         0.8,
29         (0, 0, 255),
30         1,
31         cv2.LINE_AA
32     )
33
34 cv2.imshow("src", src)
35 cv2.imshow("src_bin", src_bin)
36 cv2.imshow("dst", dst)
37 cv2.waitKey()
38 cv2.destroyAllWindows()
```

✓ 2.8s

Python

외곽선 검출

외곽선 검출 함수

```
cv2.findContours(image, mode, method, contours=None, hierarchy=None,
                 offset=None) -> contours, hierarchy
```

- image: 입력 영상. non-zero 픽셀을 객체로 간주함.
- mode: 외곽선 검출 모드. cv2.RETR_로 시작하는 상수.
- method: 외곽선 근사화 방법. cv2.CHAIN_APPROX_로 시작하는 상수.
- contours: 검출된 외곽선 좌표. `numpy.ndarray`로 구성된 리스트.
`len(contours)=전체 외곽선 개수(N)`.
`contours[i].shape=(K, 1, 2)`. `contours[i].dtype=numpy.int32`.
- hierarchy: 외곽선 계층 정보. `numpy.ndarray`. `shape=(1, N, 4)`. `dtype=numpy.int32`.
`hierarchy[0, i, 0] ~ hierarchy[0, i, 3]`이 순서대로 next, prev, child, parent 외곽선 인덱스를 가리킴. 해당 외곽선이 없으면 -1.
- offset: 좌표 값 이동 옵셋. 기본값은 (0, 0).

```
cv2.findContours(image, mode, method, contours=None, hierarchy=None,
                 offset=None) -> contours, hierarchy
```

- image: 입력 영상. non-zero 픽셀을 객체로 간주함.
- mode: 외곽선 검출 모드. cv2.RETR_로 시작하는 상수.
- method: 외곽선 근사화 방법. cv2.CHAIN_APPROX_로 시작하는 상수.
- contours: 검출된 외곽선 좌표. `numpy.ndarray`로 구성된 리스트.
`len(contours)=전체 외곽선 개수(N)`.
`contours[i].shape=(K, 1, 2)`. `contours[i].dtype=numpy.int32`.
- hierarchy: 외곽선 계층 정보. `numpy.ndarray`. `shape=(1, N, 4)`. `dtype=numpy.int32`.
`hierarchy[0, i, 0] ~ hierarchy[0, i, 3]`이 순서대로 next, prev, child, parent 외곽선 인덱스를 가리킴. 해당 외곽선이 없으면 -1.
- offset: 좌표 값 이동 옵셋. 기본값은 (0, 0).

외곽선 그리는 함수

```
cv2.drawContours(image, contours, contourIdx, color, thickness=None,
                 lineType=None, hierarchy=None, maxLevel=None, offset=None)
-> image
```

- image: 입출력 영상
- contours: (cv2.findContours() 함수로 구한) 외곽선 좌표 정보
- contourIdx: 외곽선 인덱스. 음수(-1)를 지정하면 모든 외곽선을 그린다.
- color: 외곽선 색상
- thickness: 외곽선 두께. `thickness < 0`이면 내부를 채운다.
- lineType: LINE_4, LINE_8, LINE_AA 중 하나 지정
- hierarchy: 외곽선 계층 정보.
- maxLevel: 그리기를 수행할 최대 외곽선 레벨. `maxLevel = 0` 이면 `contourIdx`로 지정된 외곽선만 그린다.

외곽선 검출 예제


```

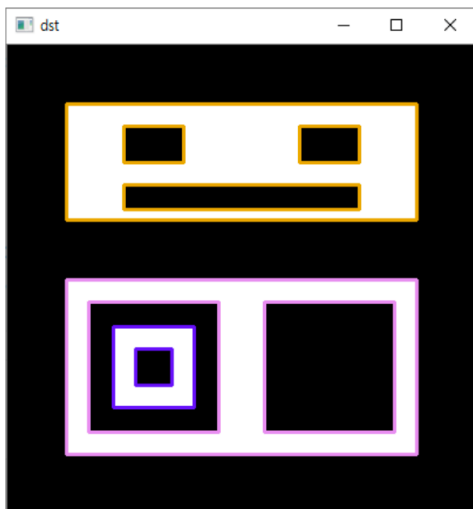
1 import sys
2 import random
3 import cv2
4
5
6 src = cv2.imread('./data/contours.jpg', cv2.IMREAD_GRAYSCALE)
7
8 if src is None:
9     print('Image load failed!')
10    sys.exit()
11
12 contours, hier = cv2.findContours(src, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_NONE)
13
14 dst = cv2.cvtColor(src, cv2.COLOR_GRAY2BGR)
15
16 idx = 0
17 while idx >= 0:
18     c = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
19     cv2.drawContours(dst, contours, idx, c, 2, cv2.LINE_8, hier)
20     idx = hier[0, idx, 0]
21
22 cv2.imshow('src', src)
23 cv2.imshow('dst', dst)
24 cv2.waitKey()
25 cv2.destroyAllWindows()

```

✓ 3.0s

Python

결과



계층 정보 없는 외곽선

```

import sys
import random
import numpy as np
import cv2

src = cv2.imread("./data/milkdrop.bmp", cv2.IMREAD_GRAYSCALE)

if src is None:
    print("Image load failed!")
    sys.exit()

_, src_bin = cv2.threshold(src, 0, 255, cv2.THRESH_OTSU)

contours, _ = cv2.findContours(src_bin, cv2.RETR_LIST, cv2.CHAIN_APPROX_NONE)

h, w = src.shape[:2]
dst = np.zeros((h, w, 3), np.uint8)

for i in range(len(contours)):
    c = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
    cv2.drawContours(dst, contours, i, c, 1, cv2.LINE_AA)

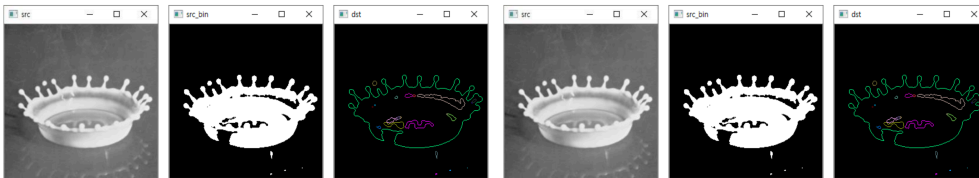
cv2.imshow("src", src)
cv2.imshow("src_bin", src_bin)
cv2.imshow("dst", dst)
cv2.waitKey()
cv2.destroyAllWindows()

```

✓ 1.1s

Python

이렇게



외곽선 관련 함수

함수 이름	설명
<code>cv2.arcLength()</code>	외곽선 길이를 반환
<code>cv2.contourArea()</code>	외곽선이 감싸는 영역의 면적을 반환
<code>cv2.boundingRect()</code>	주어진 점을 감싸는 최소 크기 사각형(바운딩 박스) 반환
<code>cv2.minEnclosingCircle()</code>	주어진 점을 감싸는 최소 크기 원을 반환
<code>cv2.minAreaRect()</code>	주어진 점을 감싸는 최소 크기 회전된 사각형을 반환
<code>cv2.minEnclosingTriangle()</code>	주어진 점을 감싸는 최소 크기 삼각형을 반환
<code>cv2.approxPolyDP()</code>	외곽선을 근사화(단순화)
<code>cv2.fitEllipse()</code>	주어진 점에 적합한 타원을 반환
<code>cv2.fitLine()</code>	주어진 점에 적합한 직선을 반환
<code>cv2.isContourConvex()</code>	컨벡스인지를 검사
<code>cv2.convexHull()</code>	주어진 점으로부터 컨벡스 헐을 반환
<code>cv2.convexityDefects()</code>	주어진 점과 컨벡스 헐로부터 컨벡스 디펙트를 반환

외곽선 길이 구하기

```
cv2.arcLength(curve, closed) -> retval
```

- curve: 외곽선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- closed: True이면 폐곡선으로 간주
- retval: 외곽선 길이

면적 구하기

```
cv2.contourArea(contour, oriented=None) -> retval
```

- contour: 외곽선 좌표. `numpy.ndarray`. `shape=(K, 1, 2)`
- oriented: True이면 외곽선 진행 방향에 따라 부호 있는 면적을 반환. 기본값은 False.
- retval: 외곽선으로 구성된 영역의 면적

바운딩 박스 구하기

```
cv2.boundingRect(array) -> retval
```

- array: 외곽선 좌표. `numpy.ndarray.shape=(K, 1, 2)`
- retval: 사각형 정보. (x, y, w, h) 튜플.

바운딩 서클 구하기

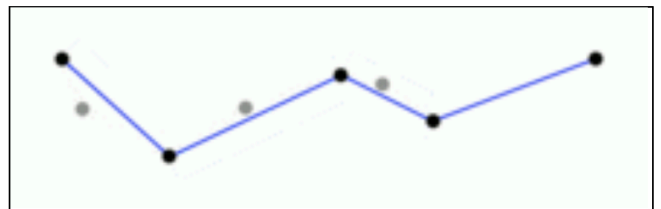
```
cv2.minEnclosingCircle(points) -> center, radius
```

- points: 외곽선 좌표. `numpy.ndarray.shape=(K, 1, 2)`
- center: 바운딩 서클 중심 좌표. (x, y) 튜플.
- radius: 바운딩 서클 반지름. 실수.

외곽선 근사화

```
cv2.approxPolyDP(curve, epsilon, closed, approxCurve=None) -> approxCurve
```

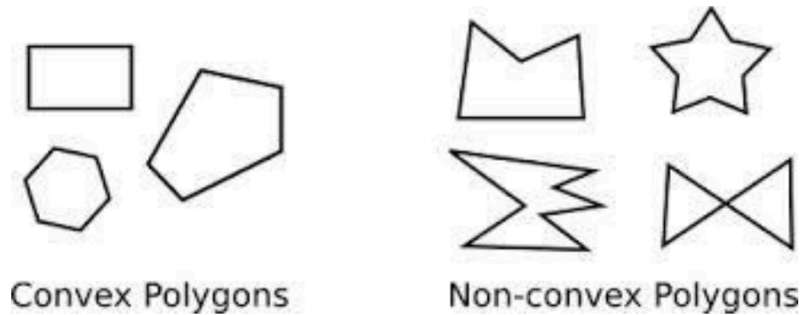
- curve: 입력 곡선 좌표. `numpy.ndarray.shape=(K, 1, 2)`
- epsilon: 근사화 정밀도 조절. 입력 곡선과 근사화 곡선 간의 최대 거리.
e.g) `cv2.arcLength(curve) * 0.02`
- closed: True를 전달하면 폐곡선으로 인식
- approxCurve: 근사화된 곡선 좌표. `numpy.ndarray.shape=(K', 1, 2)`
- 참고사항:
 - 더글라스-포이커 알고리즘 (Douglas–Peucker algorithm) [[Wiki](#)]



컨벡스 검사

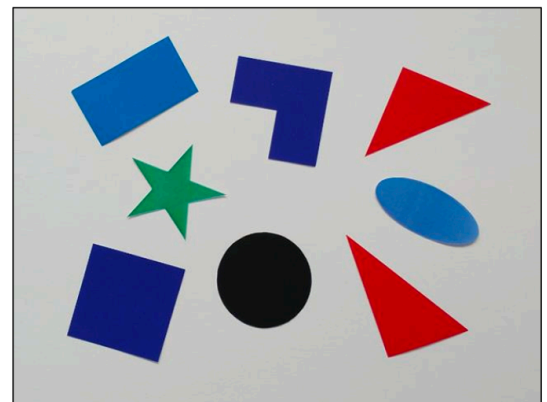
```
cv2.isContourConvex(contour) -> retval
```

- contour: 입력 곡선 좌표. `numpy.ndarray.shape=(K, 1, 2)`
- retval: 컨벡스이면 True, 아니면 False.



다각형 검출해보기

- 다각형 판별 프로그램
 - 다양한 다각형 객체 영상에서 삼각형, 사각형, 원 찾기
- 구현 순서
 - 1) 이진화
 - 2) 외곽선 찾기
 - 3) 외곽선 근사화
 - 4) 너무 작은 객체와 컨벡스가 아닌 객체 제외
 - 5) 꼭지점 개수 확인
 - 1) 삼각형, 사각형 검출
 - 2) 원 판별



원 판별하기

- 정해진 외곽선 길이에 대한 넓이 비율이 가장 큰 형태가 원
→ 도형의 넓이(A)와 외곽선 길이(P)의 비율을 검사

$$\frac{A}{P} = \frac{\pi r^2}{2\pi r} \rightarrow \frac{A}{P^2} = \frac{\pi r^2}{4\pi^2 r^2} = \frac{1}{4\pi} \rightarrow 4\pi \frac{A}{P^2} = 1$$

- $4\pi \frac{A}{P^2}$ 값이 1에 가까울수록 원으로 판단

```
length = cv2.arcLength(pts, True)
area = cv2.contourArea(pts)
ratio = 4. * math.pi * area / (length * length)

if ratio > 0.85:
    setLabel(img, pts, 'CIR') # 원
```

코드

```

import math
import cv2

def setLabel(img, pts, label):
    (x, y, w, h) = cv2.boundingRect(pts)
    pt1 = (x, y)
    pt2 = (x + w, y + h)
    cv2.rectangle(img, pt1, pt2, (0, 0, 255), 1)
    cv2.putText(img, label, pt1, cv2.FONT_HERSHEY_PLAIN, 1, (0, 0, 255))

def main():
    img = cv2.imread("./data/polygon.jpg", cv2.IMREAD_COLOR)
    if img is None:
        print("Image load failed!")
        return
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, img_bin = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)
    contours, _ = cv2.findContours(img_bin, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
    for pts in contours:
        if cv2.contourArea(pts) < 400: # 너무 작으면 무시
            continue
        approx = cv2.approxPolyDP(pts, cv2.arcLength(pts, True) * 0.02, True)
        vtc = len(approx)
        if vtc == 3:
            setLabel(img, pts, "TRI")
        elif vtc == 4:
            setLabel(img, pts, "RECT")
        else:
            length = cv2.arcLength(pts, True)
            area = cv2.contourArea(pts)
            ratio = 4.0 * math.pi * area / (length * length)
            if ratio > 0.85:
                setLabel(img, pts, "CIR")
    cv2.imshow("img", img)
    cv2.waitKey()
    cv2.destroyAllWindows()

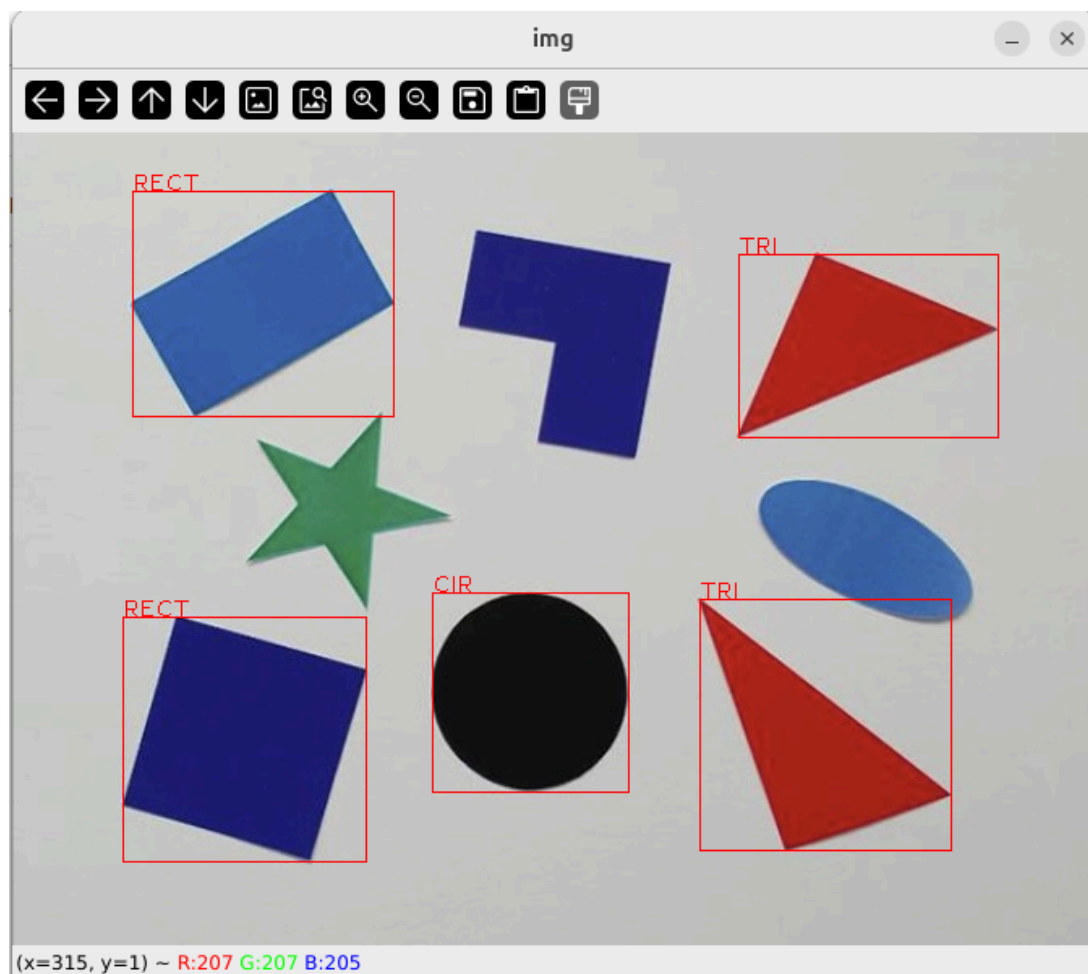
if __name__ == "__main__":
    main()

```

✓ 4.8s

Python

결과





PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio